**OOP HW**
**Stage 1**

Make a class named Order. Each order object will have a name, quantity (int), online or offline (1/0) as attributes. In addition, the order will have a string version of the online or offline attribute, with values 'Online' or 'Offline'. The string version is generated inside the class. (not in submit/process_line)

You will have a global list to store order objects.

Use the main below: (variations are okay so long as functions remain same and you do not do a *def main()*. In particular, the functions listed below do not have any parameters or return values.)

```
quit = False
while not quit:
print('1.Submit 2.Load 3. Summary 4.Display 5. Save 6.Search 7.Reset 8.Exit')
choice = int(input('Enter choice:  '))
if choice == 1:
    submit()
elif choice == 2:
    load()
elif choice == 3:
    summary()
elif choice == 4:
    display()
elif choice == 5:
    save()
elif choice == 6:
    search()
elif choice == 7:
    reset()
elif choice == 8:
    quit = True
else:
    print('Invalid Choice!')
```

**Submit**
Collects inputs  Name, Quantity, Online or Offline (must be collected as 1/0) as a single line. Submit inputs are separated by space(s), not commas.

Creates an order object.

Appends the object to a list of orders.

Prints the order.

Of these steps, submit only does two things:  Read the line from the user, and print the order.  The steps in between are done by a function called by submit,  **process_line()**

Submit output could look like this initially: (need not match exactly, you can split into multiple lines, for example; the 1/0 is not to be included in the output here)
Name: Joe  Quantity: 100  Mode: Online

Later we will include the cost here.

**Display**
Loops through the list of objects and prints each object, with data appearing as columns in a table.  A sample row below (Exact match of width/justification is not required; clean and nice presentation is required, so also is use of formatting matching type).  Add a method (function) to the class to generate such a

row for the object, and have display use this method.  Also add functions to support headers and divider lines.
|Joe  |  100  |  Online  |

*Note that the 1/0 is not included, only the string version is.*

**Save**

Prompts the user for a filename (to save to).
Saves comma separated values for each order from the list of orders.  Add a method to the class to generate a csv string for attributes of the object.  Skip the 1/0, use the string version (Online/Offline).
Displays a message for example *10 orders saved to outputs.txt*

**Load**

Prepare a text file with comma separated input lines with Name, Quantity, Online/Offline,  such as Joe,100,1
Load reads the file, converts each line to an order object (**uses process_line function)**, appends the object to the list, and displays a message such as:  3 orders loaded from inputs.txt  (Both the count and the filename are from variables, not from values typed directly into the print. At start of load, assign the filename to a variable such as *infile* or *filename*. And use a count variable that you increment through the for loop to find how many shipments were loaded. )

**Stage 2**

## Search

## Two options

1 Search by name: Collects a name from the user, and displays matching orders in the list, or Name Not Found if no match is found.
2 Search by status:  Asks user to enter 1 / 0 to search for online/offline orders, and displays matching information, or an appropriate message if no matching order is found.

**Reset**

Gets ready for a new series of inputs. Clears out data structures.

**Summary**

Summary displays:  Number of online orders, number of offline orders.  Average quantity per order,  Average quantity per online order (this is sum of online quantities / count of online orders).

The summary functionality must be implemented using class variables and class methods, using techniques similar to what was shown in lectures.  Try to use a single method to compute both averages.

Add empty list checks to all functions that retrieve data from the list. Ensure summary is just printing a report prepared by the class. Ensure the captions and dividers for display are handled by the class, as shown in lecture.

**Price computation**

Each order will have a price attribute. To support this, we need to store unit prices in class, add a compute_price method to the class, and include price in all outputs from the class.

Unit prices: store these as a class variable in the form of a dictionary as done in the last lecture. Unit price values: Online orders : 10 $. Offline orders 11 $. Your compute_price method should use unit prices from this dictionary to compute the order price, which is then set as a new attribute for the Order object. Don't forget to include this new attribute in all output methods. Use keys 'Online', 'Offline', when you store the unit prices. Reset will not affect this dictionary.

**Additional Specifications**

If needed, save before doing a reset or exit. (to cover for the possibility that the user forgot to save.)
If save is needed, ask the user - Data has not been saved. Would you like to save first? And save only on confirmation.

**Strict No On:**

- Printing from inside the class
- Accessing the list from inside the class
- Running input statements inside the class
- The class should be completely independent of the list, and user interactions.
- To the maximum extent possible, avoid accessing any attributes (of the objects or of the class - such as order.quantity or Order.count) directly from outside the class. Instead, access them through methods written inside the class.
- Do not directly access the files (load/save) from within the Order class.